



SpaceStay

Plataforma de operação de um hotel orbital

Finalidade. Este relatório técnico documenta o projeto SpaceStay, desenvolvido para a Global Solution 2026/1 dentro do tema *A Economia Espacial: soluções para o desafio da indústria espacial*. O documento apresenta a problemática endereçada, a solução proposta e o detalhamento das seis entregas que compõem o projeto, do modelo de dados à simulação de IoT.

Integrantes

Nome	RM
Vinicius Taiki	RM554226
Matheus Diniz	RM553083
Matheus Hammarstrom	RM553403
Lucas Fonseca	RM552973

Sumário

1. Introdução

- 1.1 Contexto e problemática
- 1.2 A solução: SpaceStay
- 1.3 Visão geral da arquitetura
- 1.4 Tecnologias utilizadas

2. Modelo de banco de dados

3. API REST

4. Plano de testes

5. Aplicativo mobile

6. Segurança

7. Simulação de IoT e machine learning

8. Conclusão

O código-fonte completo, os scripts e os artefatos de cada seção estão versionados no repositório do projeto, citado na conclusão.

1. Introdução

1.1 Contexto e problemática

O turismo orbital deixou de ser ficção científica. Voos suborbitais já são comercializados e estações privadas estão em desenvolvimento para receber visitantes pagantes. Surge, com isso, um novo modelo de negócio dentro da economia espacial: a **hospitalidade orbital**. E com o novo negócio vem um desafio operacional sem paralelo na hotelaria tradicional.

Um quarto de hotel na Terra é um espaço de conforto. Um quarto de hotel em órbita é uma **cápsula selada de suporte à vida**: o ar que o hóspede respira, a concentração de CO₂, a pressão, a temperatura e a umidade são mantidos por sistemas que não podem falhar em silêncio. Um problema que começa como desconforto (o CO₂ subindo aos poucos, a umidade fora da faixa) pode escalar para um incidente de segurança em questão de minutos. Operar esse ambiente exige unir duas preocupações que, na Terra, vivem separadas: a **experiência do hóspede** (reservas, atividades, conforto) e o **monitoramento contínuo do suporte à vida** de cada módulo.

A pergunta que orienta o projeto é, portanto: como operar um hotel orbital atendendo bem o hóspede e, ao mesmo tempo, vigiando o ambiente de cada cabine para que a equipe consiga agir *antes* que um desvio vire emergência? O tema se conecta a objetivos de desenvolvimento sustentável da ONU, em especial o ODS 9 (indústria, inovação e infraestrutura) e o ODS 11 (comunidades sustentáveis), ao tratar da infraestrutura de software para um novo setor.

1.2 A solução: SpaceStay

O **SpaceStay** é a plataforma de operação desse hotel orbital. Ele integra a telemetria dos sensores de cada módulo, do lado do espaço, a um aplicativo de operação e experiência, do lado da Terra, reunindo a gestão de hospedagem e o monitoramento de suporte à vida em um único sistema. A plataforma atende dois perfis de usuário:

- **Hóspedes** se cadastram, reservam uma cabine, acompanham em tempo real o ambiente da própria cabine em cartões de conforto e reservam ou cancelam excursões.
- **Equipe** (perfis admin, engineer e concierge) monitora todos os módulos em um painel de central de operações (Mission Control) e reconhece os alertas conforme acontecem.

O ponto central da solução: quando uma leitura de sensor sai da faixa segura, o sistema gera um **alerta** classificado por gravidade e a equipe é avisada na hora. É esse mecanismo que transforma dado em ação e conforto em segurança, e ele atravessa todas as seis entregas do projeto.

1.3 Visão geral da arquitetura

A solução conecta a ponta de IoT (os sensores) ao banco de dados relacional, passando pela API, e entrega tudo a um aplicativo mobile. Os dados percorrem o caminho abaixo:

```

Sensores (ESP32 simulado no Wokwi)
  | MQTT (tópico no broker HiveMQ)
  v
Node-RED --- POST /api/readings ----> API ASP.NET Core
                                         Controller -> Service -> Repository
                                         Autenticação (hash + perfis) e Alertas
                                         | EF Core (Code-First)
                                         v
                                         Banco relacional (MySQL)
                                         guests, modules, bookings,
                                         sensors, sensor_readings, alerts

Aplicativo Android (Kotlin + Compose) --- REST / Retrofit ----> API
telas de hóspede e de equipe

```

A API é o núcleo: ela aplica a regra de alerta e é indiferente à origem da leitura, seja um sensor real, o simulador da Parte 6 ou uma chamada de teste.

1.4 Tecnologias utilizadas

Todas as tecnologias estão dentro do conteúdo trabalhado no curso.

Camada	Tecnologias
Banco de dados	MySQL 8 (InnoDB, utf8mb4), modelo relacional com PK, FK, UNIQUE, CHECK, ENUM e índices
Backend e API	ASP.NET Core Web API (.NET 8), Entity Framework Core 8 (Code-First) com o provider Pomelo MySQL, injeção de dependência, camadas de serviço e repositório, Swagger
Segurança	PBKDF2 (hash de senha), JWT (autenticação) e RBAC (autorização por perfil)
Testes	xUnit (testes unitários e de ponta a ponta)
Mobile	Kotlin com Jetpack Compose (Material 3), MVVM com LiveData, Navigation Compose e Retrofit
IoT	ESP32 simulado no Wokwi, MQTT (broker HiveMQ) e Node-RED como ponte para a API
Machine learning	Python com scikit-learn (árvore de decisão)

2. Modelo de banco de dados

O modelo de dados é a fundação do SpaceStay e a fonte única de verdade para as demais camadas. O domínio é um hotel orbital em que cada módulo é uma cápsula selada de suporte à vida: o sistema gerencia as hospedagens e guarda a telemetria de cada módulo (O₂, CO₂, pressão, temperatura, umidade e água). Quando uma leitura sai da faixa segura, registra-se um alerta para a equipe. O modelo tem **nove entidades** e duas relações muitos-para-muitos (hospedagem e excursões), implementado em MySQL com integridade garantida por chaves, restrições e índices.

2.1 Diagrama entidade-relacionamento

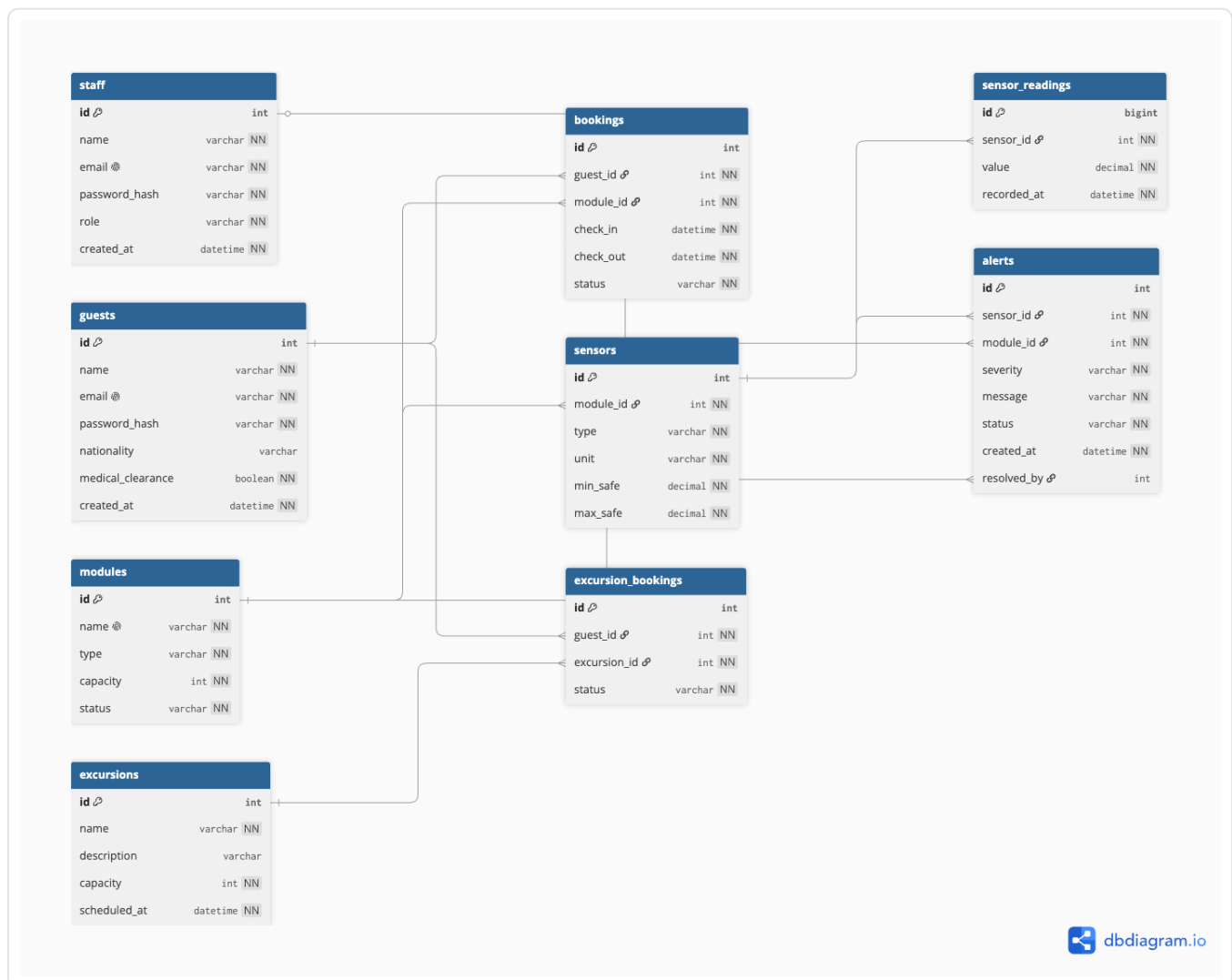


Diagrama do modelo (dbdiagram.io). PK = chave primária, FK = chave estrangeira, NN = NOT NULL, @ = UNIQUE.

2.2 Entidades

#	Tabela	Descrição	Destaques de integridade
1	guests	Hóspedes (clientes do hotel).	email único; password_hash (nunca senha pura).
2	staff	Equipe interna.	role ENUM (admin/engineer/concierge); email único.
3	modules	Módulos/quartos (cápsulas seladas).	name único; capacity > 0 (CHECK).
4	bookings	Reservas de hospedagem (N:N entre guests e modules).	FK guest (CASCADE) e module (RESTRICT); check_out > check_in (CHECK).
5	sensors	Sensores IoT por módulo.	um sensor por tipo/módulo (UNIQUE); max_safe > min_safe (CHECK).
6	sensor_readings	Leituras de telemetria (alto volume, PK BIGINT).	Índice (sensor_id, recorded_at).
7	alerts	Alertas de limite violado.	FK sensor, module e resolved_by (staff, SET NULL).
8	excursions	Atividades/excursões.	capacity > 0 (CHECK).
9	excursion_bookings	Junção N:N entre guests e excursions.	UNIQUE (guest_id, excursion_id) evita reserva duplicada.

2.3 Relacionamentos e cardinalidades

Relação	Cardinalidade	Regra de negócio
guests para bookings	1 : N	Um hóspede faz várias reservas ao longo do tempo.
modules para bookings	1 : N	Um módulo recebe várias reservas em períodos diferentes.
guests e modules	N : N (via bookings)	Hospedagem: muitos-para-muitos com período e status.
modules para sensors	1 : N	Cada módulo possui vários sensores (cadeia de telemetria IoT).
sensors para sensor_readings	1 : N	Cada sensor registra muitas leituras.
sensors para alerts	1 : N	Violação de limite gera alerta vinculado ao sensor.
modules para alerts	1 : N	Alerta também aponta o módulo (facilita o painel da equipe).
staff para alerts	1 : N (opcional)	Via resolved_by: quem tratou o alerta (pode ser nulo).
guests e excursions	N : N (via excursion_bookings)	Um hóspede reserva várias excursões e cada excursão recebe vários hóspedes.

2.4 Integridade garantida pelo schema

As regras de negócio foram levadas para dentro do banco, de modo que dados inconsistentes nem chegam a ser gravados:

- Senhas guardadas apenas como password_hash (base para a segurança da Seção 6).
- E-mail único em guests e staff (usado no login).
- Capacidade de módulo e de excursão sempre positiva (CHECK capacity > 0).
- Período de reserva coerente (CHECK check_out > check_in).
- Faixa de sensor coerente (CHECK max_safe > min_safe).
- Um sensor por tipo por módulo (UNIQUE (module_id, type)).
- Comportamento de exclusão (ON DELETE) pensado por relação: apagar um hóspede remove suas reservas em CASCADE; um módulo com reservas é protegido por RESTRICT (preserva o histórico); remover um membro da equipe deixa resolved_by em NULL (SET NULL).

2.5 Faixas seguras dos sensores

Guardadas por sensor em min_safe/max_safe e reaproveitadas pela API e pela simulação de IoT; a severidade (aviso ou crítico) é decidida na camada de serviço da API.

Sensor	Unidade	Faixa segura	Efeito ao violar
O ₂	%	19,5 a 23,5	O ₂ baixo gera alerta crítico
CO ₂	ppm	0 a 1000 (aviso acima de 1000, crítico acima de 2000)	CO ₂ alto gera aviso ou crítico
Pressão	kPa	95 a 105	fora da faixa indica possível vazamento, crítico
Temperatura	°C	18 a 27	fora da faixa afeta conforto e segurança, aviso
Umidade	%RH	30 a 60	fora da faixa favorece condensação, aviso
Água (reserva)	%	20 a 100 (aviso abaixo de 20)	reserva baixa gera aviso

2.6 Scripts e consultas

O modelo é entregue em três scripts SQL versionados:

Arquivo	Conteúdo
schema.sql	DDL das nove tabelas: PKs, FKs (com ON DELETE por relação), UNIQUE, CHECK, ENUM nativo e índices.
seed.sql	Cenário de demonstração: 5 hóspedes, 4 da equipe, 2 módulos, 12 sensores, 30 leituras, 4 alertas, 4 reservas e 7 reservas de excursão, incluindo um alerta crítico de CO ₂ em aberto.
queries.sql	Quatro consultas de simulação de uso e quatro consultas extras que alimentam o aplicativo, com explicação e resultado esperado.

Exemplos das consultas que simulam o uso real do sistema:

Alertas críticos em aberto (com o nome do módulo)

```
SELECT a.id, a.severity, a.message, a.created_at, m.name AS module_name
FROM alerts a
JOIN modules m ON m.id = a.module_id
WHERE a.status = 'open' AND a.severity = 'critical'
ORDER BY a.created_at DESC;
```

Retorna: 1 linha, CO₂ em 2150 ppm na Cupola Suite 3, nível crítico.

Última leitura de cada sensor de um módulo (telemetria atual)

```
SELECT t.sensor_id, t.type, t.unit, t.value, t.recorded_at
FROM (
  SELECT s.id AS sensor_id, s.type, s.unit, r.`value` AS value, r.recorded_at,
         ROW_NUMBER() OVER (PARTITION BY s.id ORDER BY r.recorded_at DESC) AS rn
  FROM sensors s
  JOIN sensor_readings r ON r.sensor_id = s.id
  WHERE s.module_id = 1
) AS t
WHERE t.rn = 1
ORDER BY t.type;
```

Retorna (módulo 1): co2=2150,00, humidity=46,00, o2=20,90, pressure=101,30, temperature=28,50, water=74,00.

As demais consultas cobrem hóspedes por módulo (reservas ativas), excursões por hóspede, o painel de central de operações (módulos com alertas em aberto), leituras fora da faixa segura, ocupação por módulo e alertas tratados por membro da equipe.

2.7 Validação

Os scripts foram executados em um **MySQL 8.4 real** (container Docker): o schema criou as nove tabelas, o seed carregou os dados e todas as consultas retornaram o resultado esperado. Após a carga, a base ficou com:

guests	staff	modules	bookings	sensors	sensor_readings	alerts	excursions	excursion_bookings
5	4	2	4	12	30	4	4	7

Para verificar a integridade, executamos tentativas de gravar dados inválidos; todas foram bloqueadas pelo próprio banco:

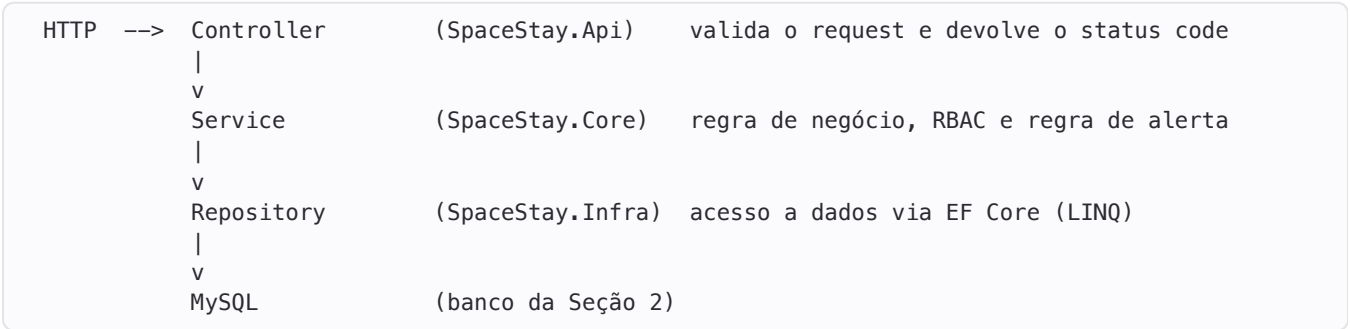
Tentativa inválida	Mecanismo	Resultado
Reserva com check_out antes do check_in	CHECK chk_bookings_dates	bloqueado (ERROR 3819)
E-mail de hóspede duplicado	UNIQUE uq_guests_email	bloqueado (ERROR 1062)
Sensor com faixa invertida (min > max)	CHECK chk_sensors_range	bloqueado (ERROR 3819)
Reserva apontando módulo inexistente	FK fk_bookings_module	bloqueado (ERROR 1452)
Papel de equipe inválido (superadmin)	ENUM role	bloqueado (ERROR 1265)
Apagar módulo que possui reservas	FK ON DELETE RESTRICT	bloqueado (ERROR 1451)
Hóspede reservando a mesma excursão duas vezes	UNIQUE uq_exbk_guest_excursion	bloqueado (ERROR 1062)

3. API REST

A API REST é a espinha dorsal do SpaceStay: ela é a ponte entre o banco de dados (Seção 2), o aplicativo mobile (Seção 5) e a simulação de IoT (Seção 7). Foi construída em **ASP.NET Core (.NET 8)** com Entity Framework Core, em arquitetura de três camadas e expondo **16 endpoints** com os verbos GET, POST, PUT e DELETE, documentados de forma interativa no Swagger.

3.1 Arquitetura em camadas

Cada requisição percorre três camadas de responsabilidades separadas, conectadas por injeção de dependência nativa. O domínio e a regra de negócio ficam isolados da infraestrutura de dados, o que torna a regra testável sem depender do MySQL (ver Seção 4).



Projeto	Responsabilidade
SpaceStay.Api	Controllers (HTTP), Program.cs (injeção de dependência, JWT, Swagger), middleware de erro, seed e serviço de JWT.
SpaceStay.Core	Domínio (entidades e enums), DTOs, interfaces e serviços com a regra de negócio.
SpaceStay.Infra	DbContext do EF Core, repositórios, migrations e o hasher PBKDF2.

3.2 Endpoints

Método	Rota	Acesso	Descrição
POST	/api/auth/register	público	Cadastro de hóspede (hash de senha)
POST	/api/auth/login	público	Login (hóspede ou equipe), devolve o JWT
GET	/api/modules	autenticado	Lista módulos (com alertas em aberto), paginado
GET	/api/modules/{id}	autenticado	Detalhe do módulo
GET	/api/modules/{id}/sensors	autenticado	Sensores e faixas seguras
GET	/api/modules/{id}/readings	autenticado	Última leitura de cada sensor
GET	/api/bookings	autenticado	Reservas (equipe: todas; hóspede: as suas)
GET	/api/bookings/{id}	autenticado	Detalhe da reserva
POST	/api/bookings	hóspede	Cria reserva (valida capacidade)
PUT	/api/bookings/{id}	autenticado	Altera ou cancela reserva
DELETE	/api/bookings/{id}	autenticado	Remove reserva
GET	/api/alerts?status=	equipe	Feed de alertas (filtro e paginação)
PUT	/api/alerts/{id}/acknowledge	equipe	Reconhece alerta (resolved_by)
POST	/api/readings	público*	Ingestão de IoT, aplica a regra de limiar de alerta
GET	/api/excursions	autenticado	Lista excursões (vagas)
POST	/api/excursions/{id}/book	hóspede	Reserva excursão (valida capacidade)

* POST /api/readings é aberto para o simulador de IoT da Seção 7. Em produção, o dispositivo usaria uma chave própria (ver Seção 6).

3.3 A regra central: ingestão de leitura e geração de alerta

O endpoint POST /api/readings é onde o IoT vira ação. No ReadingService, a API resolve o sensor e sua faixa min_safe/max_safe, persiste a leitura e, se o valor estiver fora da faixa, calcula a gravidade e cria um alerta open com uma mensagem legível. A classificação fica concentrada em AlertRules: CO₂ acima de 2000 é crítico e acima de 1000 é aviso; O₂ e pressão fora da faixa são críticos; os demais geram aviso. A equipe acompanha em GET /api/alerts?status=open e trata em .../acknowledge.

3.4 Contrato, validação e documentação

A API usa **DTOs** com validação por *data annotations* ([Required], [EmailAddress], [Range]) e devolve os códigos de status adequados a cada situação:

Status	Quando
200 / 201 / 204	Leitura, criação e remoção bem-sucedidas.
400 Bad Request	Entrada inválida (rejeitada pela validação do DTO).

Status	Quando
401 Unauthorized	Sem token ou token inválido.
403 Forbidden	Perfil sem permissão ou acesso a dado de outro usuário.
404 Not Found	Recurso inexistente.
409 Conflict	Regra de negócio violada (módulo lotado ou em manutenção, excursão duplicada).

A documentação interativa fica no **Swagger**, em /swagger, com o botão *Authorize* para informar o JWT. A persistência usa EF Core Code-First: a configuração Fluent do ApplicationDbContext espelha o schema.sql da Seção 2 (ENUM nativo, tamanhos de VARCHAR, índices, UNIQUEs, CHECKs e ON DELETE), e as migrations recriam exatamente aquele schema. Ao iniciar, a API roda Database.Migrate() e, se o banco estiver vazio, popula o cenário de demonstração.

4. Plano de testes

A qualidade da regra de negócio é sustentada por uma suíte de testes automatizados em **xUnit (.NET 8)**, com **46 testes, todos passando**. Os testes se dividem em dois níveis: **unitários**, que exercitam a regra de negócio isolada com repositórios *fake* em memória (sem banco), e **de ponta a ponta (E2E)**, que sobem a API real com TestServer e verificam a entrada e a saída de cada endpoint via HTTP.

```
tests/SpaceStay.Tests/  
  Unit/   regra isolada: AlertRules, Auth, Booking, Reading, Pbkdf2 + repositórios fake  
  E2E/    sobe a API real (SpaceStayWebAppFactory) e checa os endpoints via HTTP
```

4.1 Casos de teste

O plano cobre cenários de sucesso e de falha, com a entrada e a saída esperada de cada um:

#	Cenário	Entrada	Saída esperada
TC1	Cadastro de hóspede com sucesso	nome/e-mail/senha válidos	201, senha armazenada com hash
TC2	Login com senha errada	e-mail correto, senha errada	401, sem token
TC3	Reserva em módulo disponível	hóspede válido, módulo com vaga	201, reserva criada
TC4	Reserva em módulo lotado	módulo na capacidade máxima	409, reserva rejeitada
TC5	Leitura dentro da faixa, sem alerta	CO ₂ = 600 ppm (seguro)	201, nenhum alerta criado
TC6	Leitura fora da faixa gera alerta	CO ₂ = 2150 ppm (acima de 2000)	201, alerta crítico (open)
TC7	Hóspede em rota de equipe ou dado de outro	token de hóspede	403 Forbidden
TC8	Valor-limite: CO ₂ exatamente no máximo	CO ₂ = 1000 (igual ao limite)	seguro (sem alerta)

Além desses, a suíte cobre validação de DTO (400), acesso sem token (401), módulo em manutenção (409), excursão duplicada (409), CO₂ entre 1000 e 2000 como aviso, O₂ e pressão fora da faixa como crítico, o salt aleatório no hash e o fluxo E2E completo (cadastro, login, módulos, reserva, leitura e alerta).

4.2 Técnicas de teste aplicadas

- **Análise de valor-limite (boundary):** em torno do limite de CO₂, com 1000 (seguro), 1001 (aviso), 2000 (aviso) e 2001 (crítico); e nos limites de O₂ (19,5 seguro, 19,4 crítico).
- **Classes de equivalência:** uma representante de cada faixa (dentro, abaixo e acima) por tipo de sensor.
- **Test doubles:** repositórios *fake* em memória isolam a regra de negócio do banco nos testes unitários.

4.3 Execução

A saída da última execução está registrada em `tests/evidence/test-run.txt` (com o arquivo `.trx` ao lado):

```
SpaceStay - Evidência de execução dos testes
=====
[PASS] Booking_em_modulo_disponivel_retorna_201
[PASS] Booking_em_modulo_lotado_retorna_409
[PASS] Guest_em_rota_de_equipe_retorna_403
[PASS] Login_com_senha_errada_retorna_401
[PASS] Reading_critica_cria_alerta_critico
[PASS] Reading_dentro_da_faixa_ao_cria_alerta
[PASS] Register_retorna_201_com_token
[PASS] Evaluate_classifica_corretamente(type: co2, value: 1000, esperado: "safe")
[PASS] Evaluate_classifica_corretamente(type: co2, value: 2001, esperado: "critical")
[PASS] Evaluate_classifica_corretamente(type: o2, value: 19,4, esperado: "critical")
... (demais casos unitários: hasher, auth, booking e reading)
=====
Total: 46 | Passed: 46 | Failed: 0
```

Há ainda um teste de fumaça por `curl` em `backend/test-api.sh` (21 casos), usado em demonstrações manuais.

5. Aplicativo mobile

O aplicativo é um app Android nativo escrito em **Kotlin com Jetpack Compose** (Material 3), que consome a API REST da Seção 3. Ele tem **três telas** cobrindo os dois perfis de usuário e segue a arquitetura **MVVM com LiveData**, navegação com Navigation Compose e rede com Retrofit (um interceptor do OkHttp anexa o token JWT a cada chamada).

5.1 Arquitetura

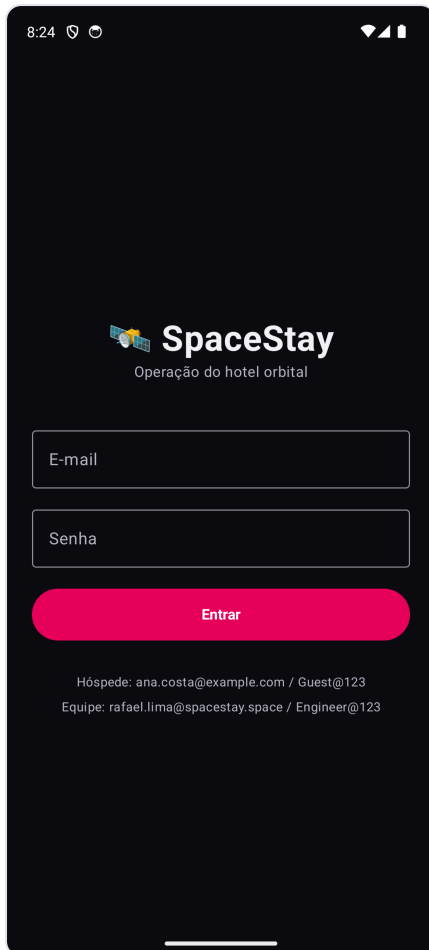
```
Composable observa -> ViewModel (LiveData) -> Repository -> Retrofit (ApiService) -> API
o token JWT fica na Session e é anexado por um interceptor do OkHttp
```

O Composable observa o estado exposto pelo ViewModel como LiveData; o ViewModel usa o Repository, que fala com a API pelo Retrofit. Cada tela trata os estados de **carregamento, erro e vazio** e usa LazyColumn/LazyRow para um layout responsivo.

5.2 As três telas

1. **Login.** Valida e-mail e senha; ao autenticar, encaminha o usuário para a área de hóspede ou de equipe conforme o userType presente no token.
2. **Hóspede, "Minha Estadia".** Mostra o módulo da reserva ativa, os medidores de conforto (O₂, CO₂, pressão, temperatura, umidade e água) com barra dentro da faixa segura e rótulo (Ideal, Atenção ou Crítico), e a lista de excursões com botão Reservar.
3. **Equipe, "Mission Control".** Apresenta os módulos em LazyRow (cor por status e número de alertas) e o feed de alertas em aberto em LazyColumn, cada um com o botão Reconhecer (que chama o acknowledge da API).

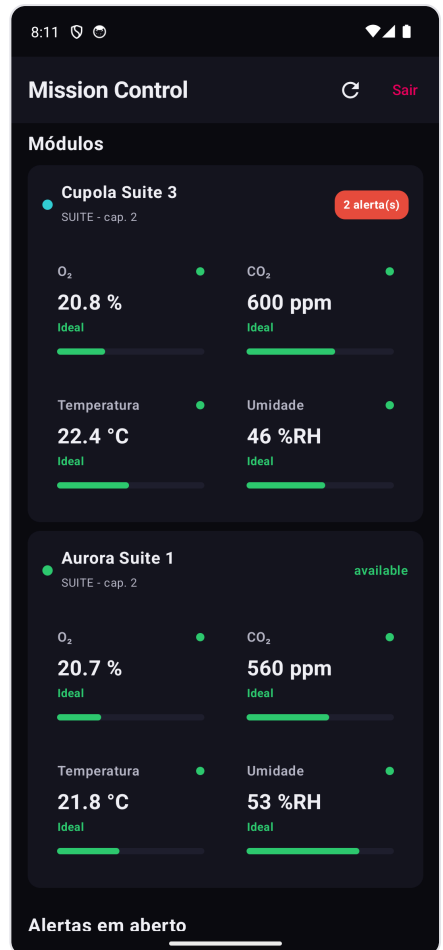
5.3 As telas em execução



Login (com validação)



Hóspede, Minha Estadia



Equipe, Mission Control

5.4 Integração com a API

O app consome os endpoints da Seção 3 para login, módulos, leituras, alertas, excursões e reservas. No emulador, a API é acessada pela URL `http://10.0.2.2:5080` (já configurada em `data/ApiClient.kt`); para um device físico, basta trocar a `BASE_URL`. A injeção de dependência é manual e simples, e o estado é gerenciado por `LiveData`, conforme o escopo trabalhado no curso.

6. Segurança

A segurança do SpaceStay se apoia em três pilares: **senha protegida por hash**, **controle de acesso por perfil** e um conjunto de **práticas adicionais** de defesa em profundidade. Toda a camada está implementada na API da Seção 3.

6.1 Senha protegida por hash

A senha nunca é armazenada em texto puro: guardamos apenas o *hash* nas tabelas `guests` e `staff` (coluna `password_hash`). O algoritmo é o **PBKDF2 com HMAC-SHA256, 100.000 iterações**, com um salt aleatório de 128 bits por usuário e chave derivada de 256 bits. O formato persistido é `pbkdf2$<iterações>$<saltBase64>$<hashBase64>`; como o salt é único, dois usuários com a mesma senha têm hashes diferentes, o que neutraliza ataques por *rainbow table*. A verificação usa comparação em **tempo constante** (`CryptographicOperations.FixedTimeEquals`), que não revela acerto pelo tempo de resposta (defesa contra *timing attack*).

6.2 Controle de acesso por perfil (RBAC)

A autenticação é feita por **JWT assinado** (HMAC-SHA256): o login emite o token com as *claims* do usuário e a API valida emissor, audiência, validade e assinatura, com expiração curta (120 minutos). A autorização acontece em **duas camadas**, seguindo o princípio do menor privilégio:

1. **No endpoint**, com os atributos `[Authorize]` e `[Authorize(Roles=...)]`: apenas a equipe lista e reconhece alertas; apenas o hóspede cria reserva ou excursão; listar módulos e reservas exige somente estar autenticado.
2. **No serviço**, checando a posse do recurso: o hóspede só enxerga e abre as próprias reservas, recebendo 403 caso tente acessar dados de outro; apenas a equipe reconhece alertas.

Token ausente ou inválido resulta em 401; perfil sem permissão resulta em 403.

6.3 Práticas adicionais

Prática	Implementação
Validação de entrada	Data annotations em todos os DTOs (<code>[Required]</code> , <code>[EmailAddress]</code> , <code>[StringLength]</code> , <code>[Range]</code>); com <code>[ApiController]</code> , uma entrada inválida é rejeitada com 400 antes de chegar à regra de negócio.
Proteção contra SQL injection	Todo o acesso a dados passa por EF Core e LINQ, que geram comandos parametrizados; não há SQL montado por concatenação de strings no projeto.
Token e falha segura	Toda rota sensível exige token; o login devolve uma mensagem genérica ("E-mail ou senha inválidos") para não revelar se o e-mail existe; e o tratamento global de erros não vaza <i>stack trace</i> em respostas 500.

6.4 Modelo de ameaças

Algumas decisões de segurança foram tomadas de forma consciente para o contexto do projeto:

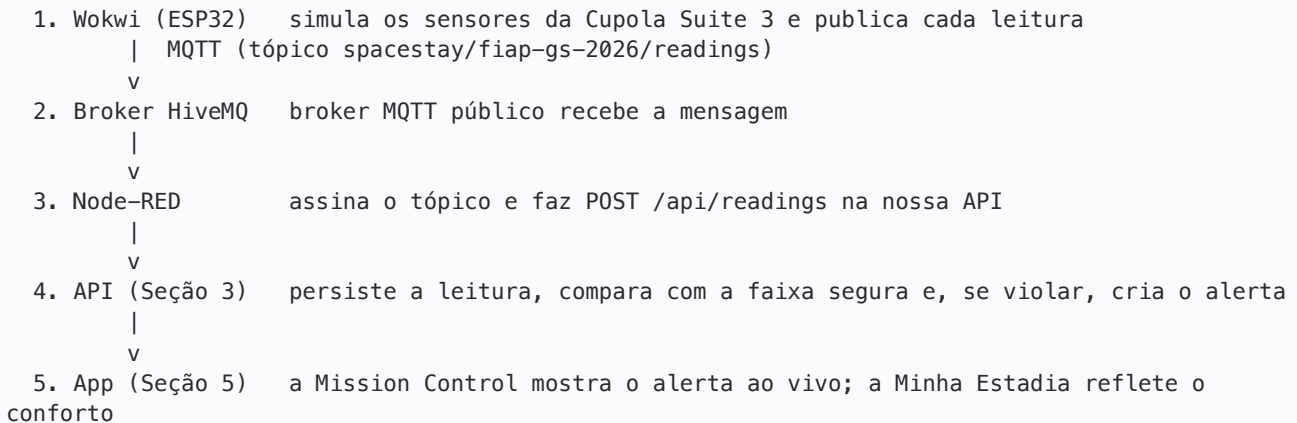
- O front-end é um app Android nativo (não um navegador), então o XSS clássico tem baixo risco; ainda assim, validamos a entrada e o Compose renderiza texto como dado, não como HTML.
- O CORS permissivo é aceitável porque CORS é um mecanismo de navegador e o cliente é nativo; em uma versão web, as origens seriam restringidas.
- O endpoint `POST /api/readings` é anônimo por decisão de projeto, para a simulação de IoT; em produção, o dispositivo usaria chave ou identidade própria.
- Os segredos (chave JWT e senha do banco) ficam em `appsettings` apenas em desenvolvimento; em produção, iriam para variáveis de ambiente ou um gerenciador de segredos.

Itens como *rate-limiting* no login, *refresh tokens*, *2FA* e *account lockout* foram deixados como evolução futura para manter o escopo enxuto.

7. Simulação de IoT e machine learning

Esta entrega fecha o ciclo entre o espaço e a Terra: sensores simulados de um módulo publicam a telemetria, que chega à API e se transforma em alerta na tela da equipe. A simulação segue a opção Arduino com MQTT, usando o **Wokwi** no lugar do TinkerCad.

7.1 Arquitetura do fluxo



A API é indiferente à origem da leitura: o contrato do endpoint é o mesmo para uma chamada de teste, para o Node-RED ou para um Arduino real.

7.2 Sensores e lógica

O ESP32 simula três sensores da Cupola Suite 3, cujos IDs vêm do `seed.sql` (Seção 2). O CO₂ é controlado por um **potenciômetro** no Wokwi (de 400 a 3000 ppm), o que permite forçar o pico durante a demonstração; a temperatura e a umidade vêm de um **DHT22** simulado.

Sensor	Faixa segura	Efeito ao violar
CO ₂ (id 2)	abaixo de 1000 ppm	acima de 1000 = aviso; acima de 2000 = crítico (problema no depurador de CO ₂)
Temperatura (id 4)	18 a 27 °C	fora da faixa = aviso (conforto e segurança)
Umidade (id 5)	30 a 60 %RH	fora da faixa = aviso (condensação)

A classificação de severidade é a **mesma da API** (concentrada em `AlertRules`): não há lógica duplicada entre o dispositivo e o servidor. O dispositivo apenas publica a leitura; quem decide se vira alerta, e com qual gravidade, é a API. Os artefatos da simulação são `iot/wokwi/sketch.ino` e `diagram.json` (sensores), `iot/node-red/flows.json` (a ponte) e `iot/run-node-red.sh` (sobe o Node-RED com o fluxo em um comando).

7.3 Validação de ponta a ponta

O caminho completo, do MQTT ao alerta na tela, foi validado: ao publicar uma leitura no tópico, o Node-RED faz o POST /api/readings e a API cria o alerta, que aparece na Mission Control. Como o contrato do endpoint é o mesmo, o caminho leitura, alerta e app também pode ser verificado com um único comando (o mesmo conteúdo que o Node-RED envia):

```
curl -X POST http://localhost:5080/api/readings \  
  -H "Content-Type: application/json" \  
  -d '{"sensorId":2,"value":2200}'  
# Resposta: 201 com alertCreated: true e severity: critical
```

7.4 Machine learning: manutenção preditiva de CO₂

Como complemento opcional, aplicamos um modelo de machine learning ao domínio. Em vez de apenas reagir quando o CO₂ estoura, a ideia é **prever** qual módulo está tendendo a estourar e sinalizar uma manutenção preventiva antes do alerta crítico. É uma análise offline (a API não chama o modelo em tempo real), feita com uma **árvore de decisão** do scikit-learn, no notebook `ml/predicao-manutencao.ipynb`.

Etapa	Detalhe
Features (tendência)	co2_atual (última leitura), co2_slope (quão rápido o CO ₂ sobe) e min_acima_800 (há quanto tempo está acima de 800 ppm).
Rótulo	vai_estourar = 1 se o módulo deve passar de 2000 ppm em breve.
Modelo	DecisionTreeClassifier(max_depth=3), com divisão treino/teste de 75/25.
Aplicação	Risco por módulo: no exemplo, a Cupola (CO ₂ subindo rápido) cai em "manutenção preventiva" e a Aurora (estável) fica "ok".

Com três features simples de tendência, a árvore já separa os módulos que caminham para um problema dos que estão estáveis. Em produção, essas mesmas features sairiam da tabela `sensor_readings` (Seção 2) exportada em CSV.

8. Conclusão

O SpaceStay foi construído em torno de um único fluxo coerente, do sensor do lado do espaço até a operação do lado da Terra, e as seis entregas se encaixam nesse fluxo. O **modelo de dados** é a fonte de verdade que guarda módulos, sensores, leituras e alertas; a **API** aplica a regra que transforma uma leitura fora da faixa em alerta e é protegida pela camada de **segurança**; o **plano de testes** garante que essa regra se comporta como o esperado nos valores-limite; o **aplicativo mobile** entrega tudo ao hóspede e à equipe; e a **simulação de IoT**, com o complemento de **machine learning**, fecha o ciclo alimentando o sistema com dados reais de telemetria.

O resultado é uma prova de conceito completa de uma plataforma de hospitalidade orbital: ela une, em um só produto, a experiência do hóspede e a vigilância do suporte à vida, mostrando como a engenharia de software pode dar suporte a um dos novos modelos de negócio da economia espacial.

Código e artefatos. Todo o código-fonte, os scripts SQL, o app, a simulação de IoT e o notebook de machine learning estão versionados no repositório público do projeto:
github.com/matheushammarstrom/gs-space-stay.